QuestionCraft

Phase 1 -- Deep Dive

End-to-End MLOps Pipeline for Automated Question Classification
& OBE-Compliant Test Paper Assembly
Built for RV College of Engineering (RVCE)

Institution:	RV College of Engineering, Bengaluru
Project:	QuestionCraft v1.0
Phase:	Phase 1 -- Development & Reproducibility
Tech Stack:	Python, scikit-learn, XGBoost, MLflow, DVC, NLTK
Document:	Complete File-by-File Explanation

QuestionCraft -- Phase 1 Deep Dive

> *A complete explanation of the project, its purpose, and every file in the repository.*

What Is QuestionCraft?

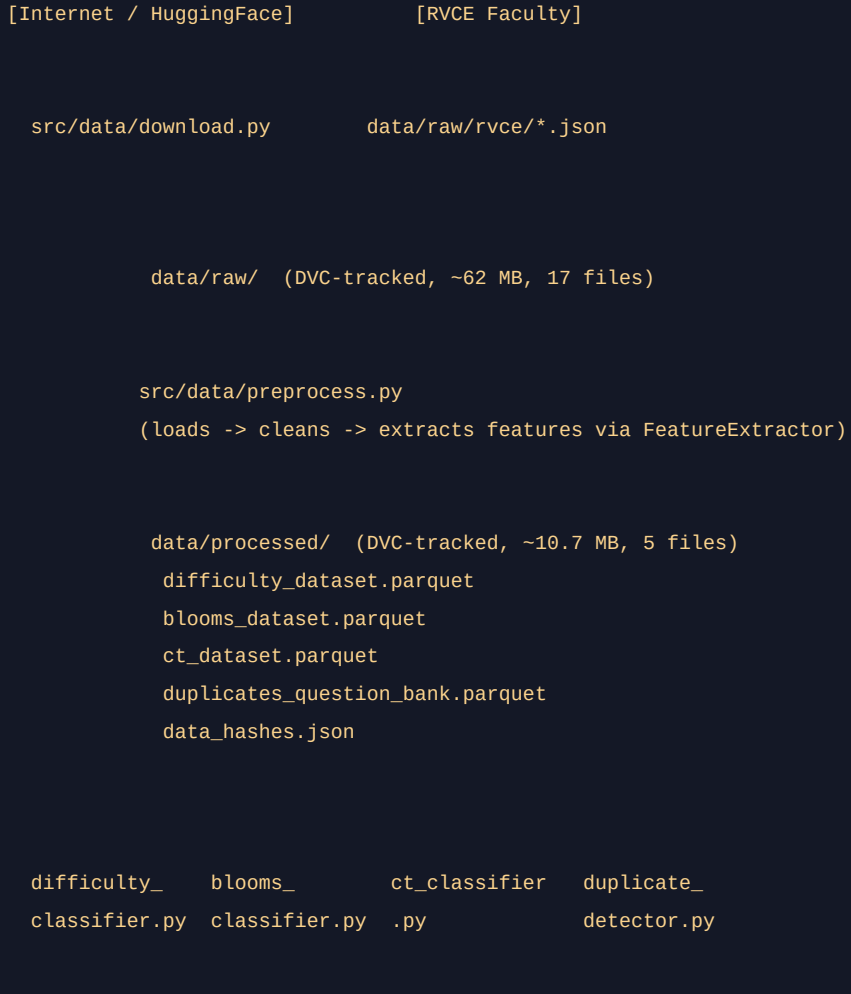
QuestionCraft is an MLOps pipeline built for RV College of Engineering (RVCE) to automate the classification of exam questions. In engineering colleges that follow Outcome-Based Education (OBE) -- required by accreditation bodies like NBA/NAAC -- every exam question must map to:

1. A difficulty level (Easy / Medium / Hard) -- so papers aren't too hard or too easy
2. A Bloom's Taxonomy level (L1 Remember -> L6 Create) -- to ensure the exam tests a range of thinking skills
3. A Computational Thinking (CT) skill (Decomposition, Pattern Recognition, etc.) -- to assess CS reasoning

Today, faculty do this tagging manually, which is slow and inconsistent. QuestionCraft trains ML models to do it automatically.

Phase 1 is entirely about building, training, and tracking those models in a reproducible, MLOps-grade way. Phase 2 (upcoming) will serve them via an API and auto-assemble exam papers.

How Data Flows Through The System



```
MLflow (mlflow.db)
models/*.pkl
```

File-by-File Explanation

[pyproject.toml]

What it is: The project's "passport" -- defines the package name, version, Python version requirement, and all dependencies with exact pinned versions.

Why it matters: By pinning exact versions (e.g., [scikit-learn==1.4.2], [mlflow==2.13.0]), anyone who clones this repo gets the identical software environment. This is critical for reproducible ML results.

Key sections:

- [[dependencies]] -- Core runtime packages: scikit-learn, XGBoost, PyTorch, sentence-transformers, MLflow, DVC, NLTK, textstat
- [[project.optional-dependencies] dev] -- Extras for development: Jupyter, matplotlib, black (formatter), ruff (linter)
- [[tool.questioncraft]] -- Custom section declaring [random_seed = 42] for numpy/torch as documentation
- [[tool.ruff]] / [[tool.black]] -- Code style configs ensuring consistent formatting

How to install: [uv pip install -e ".[dev]"]

[configs/model_config.yaml]

What it is: A single YAML file that controls every tunable parameter in the entire project -- no hardcoded values anywhere in the code.

Why it matters: If you change a hyperparameter here, DVC automatically detects the change and re-runs only the affected pipeline stages. MLflow also records the exact values from this file for each experiment run.

Sections breakdown:

Section	What it configures
`global`	`random_seed: 42`, `test_size: 0.2`, `mlflow_tra..
`tfidf`	Shared TF-IDF vectorizer settings: `max_features..
`difficulty_classifier`	Labels `[Easy, Medium, Hard]`, LR hyperparams (C..
`blooms_classifier`	Labels `[L1-Remember ... L6-Create]`, LR hyperpa..
`ct_classifier`	5 CT skill labels, OvR LR config, `similarity_th..
`duplicate_detector`	`model_name: all-MiniLM-L6-v2`, `cosine_threshol..
`feature_engineering`	Lists of Bloom's action verbs per level (L1-L6),..
`data`	Paths to raw/processed dirs, HuggingFace dataset..

[dvc.yaml]

What it is: The DVC pipeline definition -- tells DVC what commands to run, what they depend on, and what they produce.

Why it matters: It's the backbone of reproducibility. Run [dvc repro] and DVC will re-run only the stages whose inputs have changed.

The 4 stages defined:

```
train_difficulty:
  cmd: python -m src.models.difficulty_classifier --config configs/model_config.yaml ...
  deps: [difficulty_classifier.py, data/processed/, model_config.yaml]
  outs: [models/difficulty_classifier.pkl]
  params: [tfidf.*, difficulty_classifier.*]

train_blooms:  -> produces models/blooms_classifier.pkl
train_ct:      -> produces models/ct_classifier.pkl
detect_duplicates: -> produces models/duplicates_question_bank.parquet
```

How DVC detects changes: It compares MD5 hashes of every dep file. If any dep changes (code, data, or config), that stage is re-run.

[dvc.lock]

What it is: An auto-generated snapshot -- DVC writes this after each successful [dvc repro]. It records the MD5 hash of every input and output at the time the pipeline last ran.

Why it matters: This is the "proof of work". Committing [dvc.lock] means anyone who checks out this commit can verify the exact data + code + config that produced the models. It's the DVC equivalent of a [poetry.lock] / [package-lock.json].

What's in it:

- For each stage: the md5 of source files, config files, input data directories, and output [.pkl] files
- Example: [models/difficulty_classifier.pkl] has md5 [f6c4408f...], size [1,265,524 bytes]

[data/raw.dvc]

What it is: A tiny pointer file (7 lines) that tells DVC where the raw data lives in its cache and how big it is.

```
outs:
- md5: 2be67997c625406892e8aeaca78ae150.dir
  size: 62352557   # ~62 MB
  nfiles: 17
  path: raw
```

Why it matters: The actual raw data files are NOT in Git (they're in the DVC cache or remote). This pointer is what goes into Git instead. Anyone cloning the repo can run [dvc pull] to download the exact 62 MB of data.

[data/processed.dvc]

Same concept as [raw.dvc] but for the processed Parquet files (~10.7 MB, 5 files). These are the ready-to-train datasets with features already extracted.

[data/.gitignore]

Auto-generated by DVC. Tells Git to ignore the actual [data/raw/] and [data/processed/] directories (since DVC manages them), while still tracking the [.dvc] pointer files.

[src/data/] -- The Data Pipeline

[src/data/download.py]

Purpose: Downloads raw question datasets from HuggingFace Hub and saves each split as a [.parquet] file.

How it works -- step by step:

1. Reads [configs/model_config.yaml] for dataset IDs and raw data directory
2. Uses HuggingFace [datasets.load_dataset()] for each source:

- ARC-Challenge -> saved to [data/raw/arc/arc_challenge/{train,validation,test}.parquet]
- ARC-Easy -> saved to [data/raw/arc/arc_easy/{train,validation,test}.parquet]
- RACE -> saved to [data/raw/race/{train,validation,test}.parquet]
- SciQ -> saved to [data/raw/sciq/{train,validation,test}.parquet]
- Bloom's (HuggingFace) -> tries two candidate repos; saves to [data/raw/blooms/]

3. After downloading, computes an MD5 hash of every file
4. Writes a [data/raw/download_manifest.json] recording the seed, file paths, and hashes

Why parquet? It's a columnar binary format -- much faster to read than CSV and much smaller in size (compression is built-in).

CLI usage: [python -m src.data.download --config configs/model_config.yaml]

[src/data/preprocess.py]

Purpose: Reads raw data, normalizes it into a unified schema, calls [FeatureExtractor], and writes processed Parquet files ready for model training.

How it works -- step by step:

Step 1 -- Load RVCE data:

[load_all_rvce_banks()] reads both RVCE JSON files and handles two different JSON schemas:

- Old DSA bank: uses [blooms_level], [ct_skills] (list), [course_outcome]
- New UHV bank: uses [btl], [ct_skill] (string), [co]

It normalizes both into a single DataFrame and deduplicates on [question_text].

Step 2 -- Build 3 training datasets:

Dataset	Sources merged	Label column
`difficulty_dataset.parquet`	ARC + RACE + SciQ + Bloom's CSV + RVCE	`difficulty`
`blooms_dataset.parquet`	Bloom's HF + RVCE	`blooms_level`
`ct_dataset.parquet`	RVCE only (has CT annotations)	`ct_skills`

Step 3 -- Feature extraction:

[extract_features_with_cache()] deduplicates question_texts first (so identical questions aren't processed twice), calls [FeatureExtractor.extract_batch()] on unique questions, then merges the feature columns back in.

Step 4 -- Save outputs + manifest:

Writes each processed parquet + a [data_hashes.json] file containing MD5 hashes of every output for traceability.

[src/data/feature_engineering.py]

Purpose: The [FeatureExtractor] class -- the NLP heart of the project. Converts a raw question string into a fixed-length numerical feature vector.

Why features? TF-IDF captures which words appear. These hand-crafted features capture how the question is written -- complementary signals.

Features extracted (30+):

Readability metrics (8) -- via the [textstat] library:

- [flesch_reading_ease] -- Higher = easier to read (60-70 is "standard")
- [coleman_liau_index], [automated_readability_index] -- Grade-level estimates
- [dale_chall_readability_score] -- Based on a list of "easy" common words
- [flesch_kincaid_grade], [smog_index], [gunning_fog], [linsear_write_formula]

Structural features (10):

- [num_words], [num_sentences], [num_chars]
- [avg_word_length], [max_word_length], [avg_sentence_length]
- [num_question_marks] -- Does the question even end with [?]?
- [num_code_backticks] -- Are there code snippets?
- [has_numeric] -- Does it involve numbers/calculations?
- [num_uppercase_words] -- Acronyms like "CPU", "RAM"?

POS-tag features (5) -- via NLTK:

- Fraction of tokens that are nouns (NN), verbs (VB), adjectives (JJ), adverbs (RB), wh-words (WP)
- e.g., "Analyze the algorithm" has high VB fraction -> likely L4 Bloom's

Bloom's verb indicators (12) -- 2 per level (hit count + binary flag):

- L1: [define, list, recall, identify, name, state, label, recognize]
- L2: [explain, describe, summarize, interpret, classify, compare]
- L3: [apply, solve, compute, execute, implement, use, demonstrate]
- ... up to L6: [create, design, construct, develop, formulate, generate]

CT keyword indicators (10) -- 2 per skill (hit count + binary flag):

- Decomposition: [break, sub-problem, divide, decompose, modularize, partition]
- Pattern Recognition: [pattern, trend, similarity, recurring, sequence]
- Abstraction: [abstract, generalize, model, simplify, represent, hide, interface]
- Algorithmic Thinking: [algorithm, step, procedure, flowchart, pseudocode, iterate, loop, sort]
- Evaluation & Debugging: [debug, trace, error, test, evaluate, optimize, refine, fix, verify]

Key methods:

- [extract(text)] -> single question -> dict of features
- [extract_batch(texts, n_jobs=-1)] -> parallelized batch extraction using [joblib]
- [feature_names] property -> list all feature names (useful for inspection)

[src/models/] -- The ML Models

All four model scripts share the same structure:

1. Load config -> load processed data -> set up MLflow experiment -> train -> evaluate -> log everything -> save model

---

[src/models/difficulty_classifier.py]

Task: Given a question text, predict Easy / Medium / Hard.

Algorithm -- TF-IDF + Logistic Regression (baseline):

- TF-IDF (Term Frequency-Inverse Document Frequency): Converts each question into a sparse vector of 20,000 features. The value for each word/bigram is high if that term appears often in this question but rarely across all questions. Captures distinctive vocabulary. [sublinear_tf=True] uses log(TF) to dampen the effect of very frequent words.
- Logistic Regression: A linear classifier that learns a weight for each TF-IDF feature per class. [class_weight="balanced"] handles class imbalance by upweighting underrepresented classes.
- Pipeline: The two steps are wrapped in a [sklearn.Pipeline] so they're treated as a single estimator (important for correct cross-validation -- TF-IDF is fit only on training data, not leaking test data into vocabulary).

Algorithm -- XGBoost (improved, optional):

- [LabelEncodedXGBClassifier] is a wrapper that converts string labels (Easy/Medium/Hard) to integers (0/1/2) for XGBoost and decodes predictions back to strings.
- Uses gradient-boosted decision trees -- often outperforms LR on structured/mixed feature data.

Evaluation:

- 5-fold Stratified Cross-Validation on training set -> logs [cv_f1_macro_mean +/- cv_f1_macro_std]
- Held-out test set (20%) -> logs accuracy, F1-macro, precision-macro, recall-macro, plus per-class metrics
- Confusion matrix saved as PNG and logged as MLflow artifact

MLflow logging: Every run logs [data_hash] (MD5 of the DataFrame), all hyperparams, and registers the model as [questioncraft_difficulty_classifier] in the MLflow Model Registry.

CLI: [python -m src.models.difficulty_classifier --model xgboost] (switch models without editing code)

[src/models/blooms_classifier.py]

Task: Given a question text, predict which Bloom's Taxonomy Level it tests (L1-L6).

Why it's harder than difficulty: 6 classes instead of 3, and the boundaries between L3/L4/L5 are subtle (all require higher-order thinking, just different kinds).

Label normalization: The [_BLOOMS_NORM] dict handles the messy reality that different datasets use different label formats:

- "1" / "l1" / "remember" / "knowledge" -> all map to "L1"
- "6" / "l6" / "create" / "synthesis" -> all map to "L6"

This is critical when merging the RVCE data (uses "L3-Apply") with the HuggingFace Bloom's dataset (might use "3" or "apply").

Algorithm: TF-IDF + Logistic Regression (C=0.5 -- more regularization than difficulty, because 6-class is harder to overfit away from). XGBoost fallback also configured.

Key difference from difficulty classifier: No XGBoost CLI toggle here -- it only uses LR. The confusion matrix is plotted in purple ([cmap="Purples"]) to visually distinguish it from the difficulty one (blue ["Blues"]).

[src/models/ct_classifier.py]

Task: Given a question text, predict which Computational Thinking skills it tests -- but a question can test multiple skills at once. This is a multi-label classification problem.

Why multi-label? A question like "Design an algorithm to decompose a sorting problem into subproblems" tests both Algorithmic Thinking AND Decomposition.

Algorithm -- TF-IDF + One-vs-Rest (OvR) Logistic Regression:

- MultiLabelBinarizer: Converts [["Decomposition", "Algorithmic Thinking"]] into a binary vector [[1, 0, 0, 1, 0]] (one position per CT skill).
- OneVsRestClassifier: Trains one binary LR classifier per label (5 total). Each asks "does this question test Decomposition? yes/no". All 5 fire independently.
- Probability threshold: Instead of the default 0.5, you can tune [similarity_threshold] in config to change the sensitivity. At 0.5: predict positive if the model is >50% confident.

Metrics specific to multi-label:

- Hamming Loss: Fraction of labels that are incorrectly predicted (both false positives and false negatives). Lower is better.
- F1-micro: Aggregates TP/FP/FN across all labels before computing F1. Favors labels with more samples.
- F1-macro: Averages F1 across all labels equally. Good when all CT skills matter equally.
- F1-samples: Averages F1 per sample. Tells you how well the model predicts the full label set for each question.

Saved artifact: The [.pkl] file contains a dict [{"pipeline": ..., "mlb": ..., "threshold": ...}] -- the MultiLabelBinarizer must be saved too so predictions can be decoded back to label names.

[src/models/duplicate_detector.py]

Task: Find question pairs in the bank that are semantically near-duplicate (to prevent repeated/similar questions in exams).

Why semantic (not exact string match)? "What is a stack?" and "Define the stack data structure." are different strings but mean the same thing.

Algorithm -- Sentence-BERT + Cosine Similarity:

- [all-MiniLM-L6-v2] is a lightweight sentence transformer model (~90 MB) that converts any text into a 384-dimensional vector where semantically similar texts have similar vectors.
- Embeddings are L2-normalized (unit vectors), so cosine similarity equals the dot product -- very fast to compute.
- A full pairwise similarity matrix is computed. Only the upper triangle (no self-comparisons, no duplicates) is scanned.
- Any pair with cosine similarity >= 0.85 is flagged as a duplicate.

Class [DuplicateDetector]:

- [encode(texts)] -> runs batched inference, returns numpy array of embeddings
- [find_duplicates(df)] -> returns a DataFrame of flagged pairs with columns [[id_a, question_a, id_b, question_b, cosine_similarity]]
- [get_embeddings_df(df)] -> stores the full embedding vectors for downstream use

Flexible input: Can read from:

-

- A RVCE JSON question bank [--input data/raw/rvce/question_bank.json]
- Any processed Parquet file: [--input-parquet data/processed/difficulty_dataset.parquet]

MLflow logging: Logs [duplicate_rate], [num_duplicate_pairs], [total_pairs_checked], [max_similarity], [mean_duplicate_similarity].

[notebooks/]

[notebooks/01_data_exploration.ipynb]

What it is: The Exploratory Data Analysis (EDA) notebook -- the first thing you run after downloading data, before writing any model code.

What it explores:

1. Distribution of difficulty levels across all datasets
2. Distribution of Bloom's taxonomy levels
3. Question length and structural analysis (word count distributions, sentence length)
4. Readability score analysis (Flesch scores across Easy/Medium/Hard)
5. Bloom's action verb analysis (which verbs appear at which levels)
6. RVCE curriculum mapping (which Course Outcomes are most represented)
7. Near-duplicate analysis preview
8. CT skill distribution in the RVCE question bank

Each section saves a figure to [reports/figures/].

[notebooks/generate_eda_notebook.py]

A helper script that programmatically generates the notebook structure (useful for version-controlling notebooks without messy JSON output cells cluttering Git diffs).

[reports/]

[reports/eda_report.html]

A standalone HTML version of the EDA -- all 8 figures embedded, no Jupyter required. Can be shared with non-technical stakeholders (faculty, department heads) who don't have Python installed.

It's excluded from GitHub's language detection via [.gitattributes] (otherwise GitHub would incorrectly think the repo is 90% HTML).

[reports/figures/]

8 high-resolution PNG exports of the EDA plots, one per analysis dimension. These are committed to Git so they're viewable directly on GitHub without running any code.

[models/]

Contains the trained model artifacts ([.pkl] files). These are DVC-tracked, not stored in Git directly:

- [difficulty_classifier.pkl] -- 1.27 MB sklearn Pipeline
- [blooms_classifier.pkl] -- 637 KB sklearn Pipeline
- [ct_classifier.pkl] -- 45 KB dict [{pipeline, mlb, threshold}]
- [duplicates_question_bank.parquet] -- 2.7 KB table of flagged duplicate pairs

The [.gitkeep] file is a convention to keep the empty [models/] directory tracked in Git before any models are trained.

Root-Level Config Files

[.gitignore]

Tells Git what NOT to track:

- [data/raw/], [data/processed/] -- managed by DVC
- [mlflow.db], [mlruns/] -- MLflow tracking database & artifacts
- [__pycache__/], [*.pyc] -- Python bytecode
- [.venv/], [*.egg-info/] -- virtual environments
- [tmp_artifacts/] -- temporary confusion matrix PNGs generated during training

[.dvcignore]

Like [.gitignore] but for DVC. Prevents DVC from tracking patterns that should be excluded from pipeline dependency checking.

[.gitattributes]

Overrides GitHub's language detection:

```
reports/eda_report.html linguist-generated=true
```

This prevents GitHub from counting the 600KB HTML report as the primary project language.

[dvc.pipeline.yaml]

An extended/alternative pipeline definition (more detailed version of [dvc.yaml]) used for documentation and the complete download+preprocess+train pipeline.

How Everything Connects: The Full Lifecycle

```
1. Clone repo + run: uv pip install -e ".[dev]"
2. Pull data:        dvc pull
3. Run pipeline:     dvc repro
    Stage 1: train_difficulty -> difficulty_classifier.pkl
    Stage 2: train_blooms     -> blooms_classifier.pkl
    Stage 3: train_ct         -> ct_classifier.pkl
```

```
Stage 4: detect duplicates -> duplicates_question_bank.parquet
4. View experiments: mlflow ui --port 5000
5. Compare runs:     mlflow experiments compare (accuracy, F1 across baseline vs XGBoost)
```

DVC guarantees: If you change [difficulty_classifier.py], only Stage 1 re-runs (Stages 2-4 are untouched).

MLflow guarantees: Every training run is permanently recorded with its data hash, hyperparameters, and metrics -- you can always trace which model version was best.

Summary Table

File	Layer	Role
`pyproject.toml`	Config	Dependency management & reproducibility
`configs/model_config.yaml`	Config	All hyperparameters & data paths
`dvc.yaml`	MLOps	Pipeline stage definitions
`dvc.lock`	MLOps	Snapshot of last successful pipeline run
`data/raw.dvc`	MLOps	DVC pointer to 62 MB raw data
`data/processed.dvc`	MLOps	DVC pointer to processed Parquet files
`src/data/download.py`	Data	HuggingFace dataset downloader
`src/data/preprocess.py`	Data	Raw -> cleaned + feature-enriched Parquet
`src/data/feature_engineering.py`	Data	30+ NLP features from question text
`src/models/difficulty_classifier.py`	Model	TF-IDF + LR/XGBoost, 3-class
`src/models/blooms_classifier.py`	Model	TF-IDF + LR, 6-class
`src/models/ct_classifier.py`	Model	TF-IDF + OvR LR, multi-label
`src/models/duplicate_detector.py`	Model	Sentence-BERT cosine similarity
`notebooks/01_data_exploration.ipynb`	EDA	Visual data analysis
`reports/eda_report.html`	Report	Shareable standalone EDA
`reports/figures/*.png`	Report	8 EDA charts
`models/*.pkl`	Artifacts	Trained serialized models (DVC-tracked)
`QUESTIONCRAFT.md`	Docs	Project README

QuestionCraft Phase 1 -- Development & Reproducibility | RVCE MLOps